Deployment Stamps pattern

Summarize this article for me

Deploy multiple independent copies of application components, including data stores, as a single group of resources. Each copy is called a *stamp*, or sometimes a *service unit*, *scale unit*, or *cell*. In a multitenant environment, each stamp serves a predefined number of tenants. Deploy more stamps to scale the solution almost linearly, serve an increasing number of tenants, deploy instances across multiple regions, and separate your customer data.

> **Note**
>
> For more information, see Architect multitenant solutions on Azure.

Context and problem

When you host an application in the cloud, consider the performance and reliability of your application. If you host a single instance of your solution, the following limitations might apply:

- **Scale limits:** A single instance of your application might reach natural scaling limits. For example, the services that you use might limit the number of inbound connections, host names, Transmission Control Protocol (TCP) sockets, or other resources.

- **Nonlinear scaling or cost:** Some of your solution's components might not scale linearly with the number of requests or the amount of data. Instead, performance can drop or cost can spike after you meet a threshold. For example, you might find that adding more capacity to a database, or scaling up, becomes prohibitively expensive and that scaling out is more cost effective.

- **Separation of customers:** You might need to isolate one customer's data from another customer's data. You might also have customers that consume more system resources than others. You can group them on different sets of infrastructure.

- **Single-tenant and multitenant instances:** Some large customers might need their own independent instances of your solution. Smaller customers can share a multitenant deployment.

- **Complex deployment requirements:** You might need to deploy updates to your service in a controlled manner and deploy to different subsets of your customer base at different times.

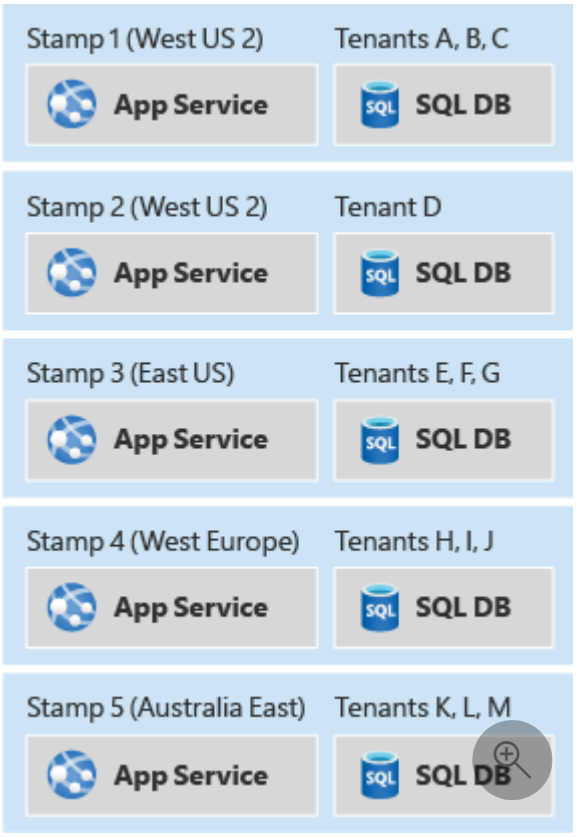
- **Update frequency:** Some customers tolerate frequent updates, while risk-averse customers want infrequent updates to the system that serves their requests. You can deploy these customers to isolated environments.

- **Geographical or geopolitical restrictions:** To achieve low latency or comply with data sovereignty requirements, you might deploy some customers to specific regions.

These limitations often apply to software development companies that build software as a service (SaaS), which they typically design as multitenant. The same limitations can also apply to other scenarios.

Solution

To avoid these problems, consider grouping resources into *scale units* and provisioning multiple copies of your *stamps*. Each scale unit hosts and serves a subset of your tenants. Stamps run independently of each other, and you can deploy and update them independently. A single geographic region might contain one stamp or multiple stamps that scale out horizontally within the region. Each stamp serves a subset of your customers.



Deployment stamps can apply whether your solution uses infrastructure as a service (IaaS) or platform as a service (PaaS) components, or a combination of both. IaaS workloads typically require more intervention to scale, so this pattern can help IaaS-heavy workloads scale out.

You can use stamps to implement deployment rings. If different customers want service updates at different frequencies, group them onto different stamps and deploy updates to each stamp at a different cadence.

Stamps run independently, so they implicitly *shard* your data. A single stamp can also use further sharding internally to scale and remain elastic.

Deploying identical copies of the same components is complex, so good DevOps practices are critical. Describe your infrastructure as code so that the deployment of each stamp is predictable and repeatable.

Deployment stamps relate to but differ from geodes. In a deployment stamp architecture, each independent instance of your system serves a subset of your customers and users. In a geode architecture, every instance can serve requests from any user, but this approach is typically more complex to design and build. You can also combine the two patterns within one solution. The traffic routing approach described later in this article is an example of such a hybrid scenario.

Problems and considerations

Consider the following points as you decide how to implement this pattern:

- **Deployment process:** When you deploy multiple stamps, automate and fully repeat your deployment processes. Use Bicep or Terraform modules to declaratively define your stamps and keep the definitions consistent.
- **Cross-stamp operations:** When you deploy your solution independently across multiple stamps, it can be hard to determine how many customers you have across all your stamps. You might need to query each stamp and aggregate the results. Alternatively, you can have all stamps publish data into a centralized data warehouse for consolidated reporting.
- **Scale-out policies:** Stamps have a finite capacity, which you can define by using a proxy metric, such as the number of tenants that you can deploy to the stamp. Monitor the available and used capacity for each stamp, and proactively deploy more stamps to direct new tenants to them.
- **Minimum number of stamps:** When you use the Deployment Stamps pattern, deploy at least two stamps of your solution. If you deploy only a single stamp, you can easily hard-code assumptions into your code or configuration that don't apply when you scale out.
- **Cost:** The Deployment Stamps pattern deploys multiple copies of your infrastructure components, which substantially increases the cost of operating your solution.
- **Moving between stamps:** Each stamp runs independently, so moving tenants between stamps can be difficult. Your application needs custom logic to transmit a customer's information to a different stamp and then remove the tenant's information from the original stamp. This process might require a backplane to communicate between stamps, which further increases the complexity of your solution.
- **Traffic routing:** As described previously in this article, routing traffic to the correct stamp for a given request can require an extra component that resolves tenants to stamps. This component might also need to be highly available.
- **Observability across stamps:** As the number of stamps increases, it becomes harder to understand overall health and detect incidents quickly. Use Azure Monitor to collect and

correlate metrics, logs, traces, and alerts across all stamps. Use this data to identify unhealthy stamps and diagnose problems.

- **Regional failure impact:** Stamps run independently, but they aren't inherently redundant across regions. If a region that hosts one or more stamps becomes unavailable, the tenants on those stamps lose access until the region recovers or you migrate the tenants to stamps in another region. To plan for this scenario, document your recovery procedures, set tenant expectations, and consider whether critical tenants need geo-redundant stamp placement.

- **Shared components:** You might have components that you can share across stamps. For example, if you have a shared single-page app for all tenants, deploy it to one region and use Azure Front Door edge caching to replicate it globally.

- **Governance and configuration drift:** As the number of stamps increases, it becomes harder to keep security policies, role-based access control (RBAC) assignments, network controls, observability settings, and service configurations consistent. Use Azure Policy to treat governance as code and continuously validate each stamp for drift to prevent inconsistent behavior and compliance gaps.

When to use this pattern

Use this pattern when:

- Your solution has natural limits on scalability. For example, if some components can't or shouldn't scale beyond a certain number of customers or requests, use stamps to scale out.

- You need to separate certain tenants from others. If security concerns prevent you from deploying some customers into a multitenant stamp, deploy them onto their own isolated stamp.

- You need to host some tenants on different versions of your solution at the same time.

- You build multiregion applications that need to direct each tenant's data and traffic to a specific region.

- You want to achieve resiliency during outages. Stamps run independently, so if an outage affects a single stamp, tenants on other stamps remain unaffected. This isolation contains the *blast radius* of an incident or outage.

This pattern might not be suitable when:

- Your solution is simple and doesn't need to scale to a high degree.
- You can scale your system out or up within a single instance, such as by increasing the size of the application layer or by increasing the reserved capacity for databases and the storage tier.
- You need to replicate data across all deployed instances. Consider the Geode pattern for this scenario.
- You only need to scale some components and not others. For example, consider whether you can scale your solution by sharding the data store instead of deploying a new copy of all the solution components.
- Your solution consists solely of static content, such as a front-end JavaScript application. Deliver this content by a Content Delivery Network.

Workload design

Evaluate how to use the Deployment Stamps pattern in a workload's design to address the goals and principles covered in the Azure Well-Architected Framework pillars. The following table provides guidance about how this pattern supports the goals of each pillar.

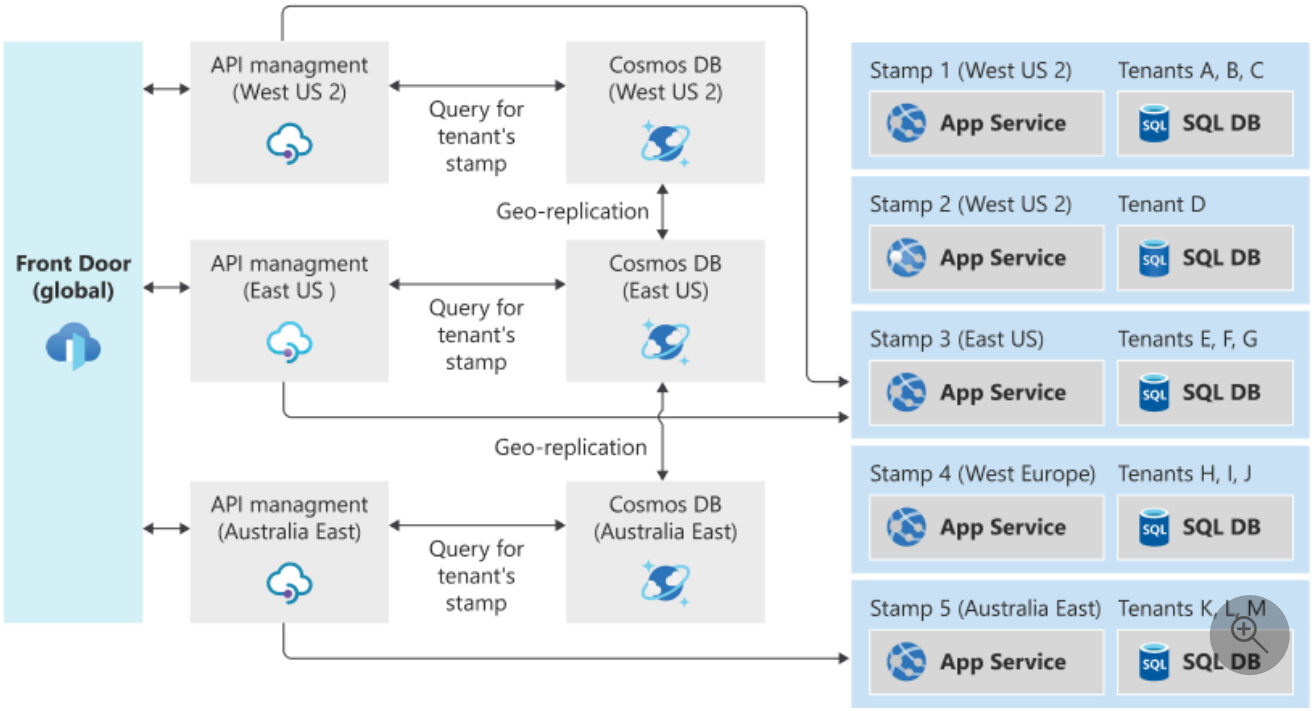
Expand table

Pillar	How this pattern supports pillar goals
Reliability design decisions help your workload become **resilient** to malfunction and ensure that it **recovers** to a fully functioning state after a failure occurs.	Stamps operate independently, so a failure in one stamp is isolated and doesn't affect tenants on other stamps. Deploying multiple stamps across regions also provides a foundation for redundancy and recovery planning, which reduces the blast radius of regional outages. - RE:05 Redundancy - RE:07 Self-preservation
Operational Excellence helps deliver **workload quality** through **standardized processes** and team cohesion.	This pattern supports immutable infrastructure goals, advanced deployment models, and can facilitate safe deployment practices. - OE:05 Infrastructure as code - OE:11 Safe deployment practices
Performance Efficiency helps your workload **efficiently meet demands** through optimizations in scaling, data, and code.	This pattern often aligns to the defined scale units in your workload. When you need more capacity than a single scale unit provides, you deploy another stamp to scale out. - PE:05 Scaling and partitioning

If this pattern introduces trade-offs within a pillar, consider them against the goals of the other pillars.

Example

The following example architecture uses Azure Front Door, Azure API Management, and Azure Cosmos DB to route traffic globally to a series of region-specific stamps.



Suppose a user resides in New York. Stamp 3, in the East US region, stores their data.

If the user travels to California and accesses the system, the system routes their connection through the West US 2 region because that region is closest to them when they make the request. However, stamp 3 must ultimately serve the request because it stores their data. The traffic routing system routes the request to the correct stamp.

Deployment

Describe your infrastructure as code, by using Bicep or Terraform. This approach ensures that the deployment of each stamp is predictable and repeatable. It also reduces the likelihood of human errors such as accidental mismatches in configuration between stamps.

You can deploy updates automatically to all stamps in parallel. Technologies like Bicep can coordinate the deployment of your infrastructure and applications. Alternatively, you might decide to gradually roll out updates to some stamps first, and then progressively to other stamps. Consider using a release management tool like Azure Pipelines or GitHub Actions to orchestrate deployments to each stamp.

Carefully consider the topology of the Azure subscriptions and resource groups for your deployments:

- Typically, a subscription contains all resources for a single solution, so consider using a single subscription for all stamps. However, some Azure services impose subscription-wide quotas. If you use this pattern to allow for a high degree of scale-out, you might need to deploy stamps across different subscriptions.

- Resource groups generally contain components that share the same life cycle. If you plan to deploy updates to all stamps at the same time, you can use a single resource group that contains all components for all stamps. Use resource naming conventions and tags to identify the components that belong to each stamp. Alternatively, if you plan to deploy updates to each stamp independently, you can deploy each stamp into its own resource group.

Capacity planning

Use load and performance testing to determine the approximate load that a given stamp can accommodate. Load metrics might be based on the number of customers or tenants that a single stamp can accommodate, or on metrics that the services in the stamp emit. Instrument each stamp so that you can measure when it approaches its capacity, and make sure that you can deploy new stamps quickly to respond to demand.

Traffic routing

The Deployment Stamps pattern works well when you address each stamp independently. For example, if Contoso deploys the same API application across multiple stamps, Contoso might use Domain Name System (DNS) to route traffic to the relevant stamp:

- `unit1.aus.myapi.contoso.com` routes traffic to stamp `unit1` within an Australian region.
- `unit2.aus.myapi.contoso.com` routes traffic to stamp `unit2` within an Australian region.
- `unit1.eu.myapi.contoso.com` routes traffic to stamp `unit1` within a European region.

In Azure, you can host these records in Azure DNS and use a consistent subdomain convention for each region and stamp. This approach maintains predictable routing and operations.

Clients are responsible for connecting to the correct stamp.

If your solution requires a single ingress point for all traffic, you can use a traffic routing service to resolve the stamp for a given request, customer, or tenant. The traffic routing service either directs the client to the relevant URL for the stamp (for example, by returning an HTTP 302 response status code), or it acts as a reverse proxy and forwards the traffic to the relevant stamp without the client being aware.

A centralized traffic routing service can be a complex component to design, especially when a solution runs across multiple regions. Consider deploying the traffic routing service into multiple regions, potentially including every region that hosts stamps, and sync the data store that maps tenants to stamps. The traffic routing component might itself be an instance of the Geode pattern.

For example, you can deploy API Management to act as the traffic routing service. API Management determines the appropriate stamp for a request by looking up data in an Azure Cosmos DB collection that stores the mapping between tenants and stamps. API Management then dynamically sets the back-end URL to the relevant stamp's API service.

To geo-distribute requests and provide geo-redundancy for the traffic routing service, deploy API Management across multiple regions and use Azure Front Door to direct traffic to the closest API Management gateway. In this topology, Azure Front Door uses origin groups, health probes, and an appropriate routing method to route requests away from unhealthy API Management regional gateways. API Management then routes to the appropriate stamp by using the tenant-to-stamp mapping and its back-end configuration (or back-end pools), including failover rules between stamp endpoints as needed. If your application isn't exposed over HTTP or HTTPS, you can use a cross-region Azure load balancer to distribute incoming calls to regional Azure load balancers. Use the global distribution feature of Azure Cosmos DB to keep the mapping information updated across each region.

If your solution includes a traffic routing service, consider whether it acts as a gateway and can perform gateway offloading for the other services, such as token validation, throttling, and authorization.

Next steps

- Azure Front Door
- Integrate Bicep with Azure Pipelines
- Integrate JSON ARM templates with Azure Pipelines

Contributors

Microsoft maintains this article. The following contributors wrote this article.

Principal author:

- John Downs | Principal Software Engineer, Azure Patterns & Practices

Other contributors:

- Federico Arambarri | Senior Software Developer, Clarius Consulting
- Daniel Larsen | Principal Customer Engineer, FastTrack for Azure
- Angel Lopez | Senior Software Engineer, Azure Patterns and Practices
- Paolo Salvatori | Principal Customer Engineer, FastTrack for Azure
- Arsen Vladimirskiy | Principal Customer Engineer, FastTrack for Azure

To see nonpublic LinkedIn profiles, sign in to LinkedIn.

Related resources

- You can use sharding as another simpler approach to scale out your data tier. Stamps implicitly shard their data, but sharding doesn't require a deployment stamp. For more information, see Sharding pattern.
- If your solution deploys a traffic routing service, you can combine the Gateway Routing and Gateway Offloading patterns to make the best use of this component.

Feedback

Was this page helpful? Yes No

Last updated on 04/30/2026